

Theory: LLM Inference Fundamentals

Q1. Why is inference memory bandwidth-bound important?

During LLM inference (particularly autoregressive text generation), the model generates one token at a time. For each new token, the GPU must:

- (i) Load the entire set of model weights (and the KV cache) from GPU high-bandwidth memory (HBM) into the on-chip compute units.
- (ii) Perform the actual matrix multiplications for that single token.

Because only *one* token's worth of activations is being computed at a time (batch size effectively small, especially for batch size 1), the amount of **compute** required is tiny relative to the amount of **data that must be moved**. In other words, for every byte of weight data fetched from memory, very few floating-point operations are performed on it before it is discarded and the next byte is needed.

Arithmetic Intensity

This trade-off is captured by *arithmetic intensity*:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs performed}}{\text{Bytes moved from memory}}$$

Modern GPUs have compute throughput (FLOPs/s) that is far larger than their memory bandwidth (bytes/s). The **roofline model** tells us that a workload is:

- **Compute-bound** if arithmetic intensity is high (e.g., large batch training, where the same weights are reused across many samples).
- **Memory (bandwidth)-bound** if arithmetic intensity is low (e.g., single-token autoregressive decoding).

LLM decoding falls into the second category: the GPU spends most of its time *waiting* for weights and KV-cache values to arrive from HBM rather than actually computing, so the compute cores sit underutilized.

Why this matters in practice

- **Latency:** The time to generate each token is dictated by $\frac{\text{Model size in bytes}}{\text{Memory bandwidth}}$, not by the raw FLOP count of the GPU. A GPU with more bandwidth (e.g., HBM3 vs. HBM2e) will generate tokens faster even if peak FLOPs are similar.
- **Hardware selection:** It explains why inference-optimized hardware emphasizes high-bandwidth memory (HBM) and large caches, while training-optimized hardware can also prioritize raw compute (FLOPs).

- **Optimization strategies:** Techniques such as quantization (reducing weight precision from FP16 to INT8/INT4), KV-cache compression, and batching multiple requests together exist specifically to reduce the number of bytes that must be streamed per useful computation, thereby increasing arithmetic intensity and reducing the bandwidth bottleneck.
- **Cost:** Since GPUs are memory-bandwidth-bound during inference, compute capacity is often left idle. This underutilization directly affects the throughput-per-dollar of serving LLMs, making memory bandwidth (not just GPU FLOPs) a key metric when provisioning infrastructure.

In short: inference is bandwidth-bound because generating each token requires re-reading the (large) model weights and KV cache from memory for a comparatively tiny amount of computation, so the speed of memory access — not compute throughput — becomes the limiting factor.

Q2. How can we calculate the overall memory required to store a model? How are LLM models represented while storing?

Memory required to store a model

The memory needed to store a model is primarily determined by its **parameter count** and the **numeric precision (data type)** used to represent each parameter.

$$\text{Model Memory (bytes)} = N_{\text{params}} \times \text{Bytes per parameter}$$

Precision / Data Type	Bits	Bytes per Parameter
FP32 (single precision)	32	4
FP16 / BF16 (half precision)	16	2
INT8 (8-bit quantized)	8	1
INT4 (4-bit quantized)	4	0.5

Example: A 7-billion-parameter model stored in FP16:

$$7 \times 10^9 \text{ params} \times 2 \text{ bytes} = 14 \times 10^9 \text{ bytes} \approx 13.04 \text{ GB}$$

Total memory during inference (not just storage)

While the question asks about *storage*, it is worth noting that the memory *required to run* the model is higher than the raw weight storage size, because of additional components:

$$\text{Total Memory} \approx \underbrace{M_{\text{weights}}}_{\text{model parameters}} + \underbrace{M_{\text{KV cache}}}_{\text{attention key/value cache}} + \underbrace{M_{\text{activations}}}_{\text{intermediate tensors}} + \underbrace{M_{\text{overhead}}}_{\text{framework/CUDA context}}$$

where the KV cache itself scales as:

$$M_{\text{KV cache}} = 2 \times n_{\text{layers}} \times n_{\text{heads}} \times d_{\text{head}} \times \text{seq_len} \times \text{batch_size} \times \text{bytes per value}$$

(the factor of 2 accounts for storing both the Key and Value tensors).

How LLMs are represented while storing

On disk, LLM weights are represented as large multi-dimensional **tensors** (arrays of numbers), organized layer by layer. Key aspects of this representation:

- **Numeric format:** Each weight is a floating-point (or quantized integer) number, most commonly stored in FP32, FP16, or BF16 for full-precision checkpoints, and INT8/INT4 for quantized/compressed checkpoints.
- **File formats:** Common serialization formats include `.safetensors` (a safe, fast, memory-mapped tensor format), PyTorch's `.bin/.pt` (pickle-based), and `GGUF/GGML` (used for quantized models in llama.cpp-style runtimes).
- **Structure:** Each tensor is tagged with a name (e.g., `model.layers.0.self_attn.q_proj.weight`) corresponding to a specific component (embedding table, attention projection matrices, feed-forward weights, layer norms, etc.), along with its shape and dtype, so the runtime can reconstruct the full computation graph on load.
- **Sharding:** For very large models, weights are split into multiple shard files (e.g., `model-00001-of-00008.safetensors`) since a single file would be too large to load or transfer efficiently.

Q3. Explain each layer of inferencing and its importance to an institution/company (in your own words)

Serving an LLM in production is not a single step — it is a **stack of layers**, each responsible for a different concern. Below is an explanation of each layer and why it matters from a business/engineering standpoint.

1. Hardware Layer (GPUs/TPUs, Memory, Networking)

This is the physical foundation: accelerators (GPUs/TPUs), high-bandwidth memory, and interconnects (NVLink, InfiniBand) that move data between chips. **Importance:** Hardware choice directly determines cost-per-token, latency, and the maximum model size an institution can serve. Poor hardware provisioning leads to either wasted spend (over-provisioning) or unusable latency (under-provisioning).

2. Model / Weights Layer

This layer holds the actual trained parameters, in a chosen precision (FP16, INT8, INT4), and the model architecture (transformer blocks, attention heads, etc.). **Importance:** Choosing the right model size and quantization level is a direct trade-off between output quality and infrastructure cost — a critical business decision for any company deploying LLMs at scale.

3. Runtime / Inference Engine Layer

Frameworks such as vLLM, TensorRT-LLM, or Hugging Face's `text-generation-inference` manage how the model actually executes: scheduling requests, batching, memory management (e.g., PagedAttention for KV cache), and kernel optimization. **Importance:** This layer is where most of the throughput and latency gains come from. Efficient batching and caching directly reduce the number of GPUs a company needs to serve the same traffic, which is a major cost lever.

4. KV-Cache / Memory Management Layer

As tokens are generated, intermediate attention states (keys and values) are cached so they don't need to be recomputed for every new token. **Importance:** Efficient KV-cache management enables longer context windows and higher concurrent user counts without linear blow-up in memory usage — essential for chat applications and document-heavy enterprise use cases.

5. Serving / Orchestration Layer

This includes load balancers, request queues, auto-scaling policies, and routing logic that decide which GPU/node handles which request. **Importance:** This layer ensures reliability (uptime), fair allocation of GPU resources across users, and the ability to scale up or down with demand — critical for maintaining SLAs (service-level agreements) with customers.

6. API / Application Layer

The outward-facing layer: REST/gRPC APIs, authentication, rate limiting, prompt templates, and safety/guardrail filters that wrap around the raw model. **Importance:** This is what the rest of the company (or external customers) actually interacts with. It enforces business logic, security, and compliance (e.g., content filtering, PII redaction) — turning a raw model into a usable, safe product.

7. Monitoring / Observability Layer

Logging of latency, token throughput, GPU utilization, error rates, and output quality metrics. **Importance:** Without this layer, a company has no visibility into cost drivers, performance regressions, or model misuse. It is essential for continuous optimization and for justifying infrastructure spend to leadership.

Why this layered view matters to an institution

Each layer represents a different point where a company can optimize for **cost**, **latency**, **reliability**, or **quality**. Understanding the full stack allows an institution to:

- Identify the actual bottleneck (e.g., is it hardware bandwidth, or inefficient batching in the runtime layer?) rather than blindly buying more GPUs.
- Make informed build-vs-buy decisions (e.g., self-host vs. use a managed API) based on which layers they have the expertise to optimize.
- Scale reliably as user demand grows, since each layer can be scaled or optimized somewhat independently.